

What is Terragrunt? Simplifying Terraform at Scale for Modern Infrastructure

Terragrunt extends Terraform's capabilities to address the challenges of managing complex infrastructure deployments across multiple environments. It provides the structure and automation needed for enterprise-scale infrastructure as code.



The Challenge with Terraform at Scale



As Terraform Deployments Grow...

Code Duplication: Copy-pasting configurations across environments leads to maintenance nightmares

State Management: Manual backend configuration for each deployment becomes error-prone

Dependency Complexity: Orchestrating infrastructure with proper sequencing gets exponentially harder

Teams need a solution that brings [structure and consistency](#) to Terraform workflows.

Enter Terragrunt: Terraform's Thin Wrapper

Terragrunt sits as a lightweight orchestration layer on top of Terraform, enhancing its capabilities without replacing its core functionality.

Thin Wrapper

Maintained by Gruntwork, Terragrunt runs Terraform commands with additional logic while preserving Terraform's strengths

DRY Principles

Follows "Don't Repeat Yourself" approach, allowing teams to define configurations once and reuse them

Orchestration

Manages dependencies and execution order, ensuring resources are created in the correct sequence



Why Use Terragrunt?

1 Keep Terraform Code DRY

Define common configurations once and reuse them across environments, reducing duplication by up to 80%

2 Environment Management

Organize code for multiple environments (dev, staging, prod) with clear inheritance patterns

3 Remote State Automation

Automatic creation and configuration of state backends like S3 buckets and DynamoDB locks

4 Dependency Handling

Define dependencies between modules for proper deployment ordering

5 Parallel Execution

Run Terraform across multiple directories simultaneously, speeding up deployments



From chaos to order: The Terragrunt advantage

Core Features Explained



Configuration Inheritance

Use `include` blocks and `find_in_parent_folders()` to create hierarchical configurations

```
include {  
  path = find_in_parent_folders()  
}
```



Dependency Management

Explicitly define dependencies between modules

```
dependency "vpc" {  
  config_path = "../vpc"  
}
```



Remote State Management

Centralize backend configuration, auto-create storage and locks

```
remote_state {  
  backend = "s3"  
  generate = {  
    path = "backend.tf"  
    if_exists = "overwrite"  
  }  
}
```



Hooks

Execute custom code before/after Terraform commands

```
before_hook "before_apply" {  
  commands = ["apply"]  
  execute = ["echo", "Starting apply"]  
}
```

Terragrunt in Action: Example terragrunt.hcl

```
# Root terragrunt.hcl (parent)
remote_state {
  backend = "s3"
  config = {
    bucket      = "my-terraform-state"
    key         =
"${path_relative_to_include()}/terraform.tfstate"
    region      = "us-west-2"
    encrypt     = true
    dynamodb_table = "terraform-locks"
  }
}

# Common input variables for all environments
inputs = {
  company = "acme"
  owner   = "devops-team"
}
```

```
# Child terragrunt.hcl (in dev/vpc/)
include {
  path = find_in_parent_folders()
}

# Module specific configuration
terraform {
  source = "tfr://registry.terraform.io/terraform-aws-
modules/vpc/aws?version=3.14.0"
}

# Module dependencies
dependency "kms" {
  config_path = "../kms"
}

# Override inputs for this module
inputs = {
  name      = "dev-vpc"
  cidr      = "10.0.0.0/16"
  environment = "development"
  kms_key_id = dependency.kms.outputs.key_id
}
```

With this structure, you can deploy entire environments with a single command: `terragrunt run-all apply`